

**КИЇВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ТАРАСА ШЕВЧЕНКА**

Факультет комп'ютерних наук та кібернетики
Кафедра інтелектуальних програмних систем

Курсова робота

за спеціальністю 121 Інженерія програмного забезпечення

на тему:

**РОЗРОБКА ВЕБСИСТЕМИ ДЛЯ АВТОМАТИЗОВАНОГО
ОБЛІКУ, АНАЛІЗУ ТА ПРОГНОЗУВАННЯ ВИТРАТ
НА ЦИФРОВІ ПІДПИСКИ**

Виконала студентка 3-го курсу
ОПП «Програмна інженерія»
Сенечко Дана Володимирівна

(підпис)

Науковий керівник:
асистент кафедри інтелектуальних програмних систем,
доктор філософії
Ількун Олександр Петрович

(підпис)

Засвідчую, що в цій курсовій роботі немає
запозичень з праць інших авторів без
відповідних посилань.

Студентка

(підпис)

РЕФЕРАТ

Обсяг роботи 30 сторінок, 2 ілюстрації, 1 таблиця, 17 джерел.

АВТОМАТИЧНА ДЕТЕКЦІЯ ПІДПИСОК, ВЕБЗАСТОСУНОК, ІМПОРТ ТРАНЗАКЦІЙ, ПРОГНОЗУВАННЯ ВИТРАТ, ШИФРУВАННЯ, API, MONOBANK, NODE.JS, TELEGRAM-БОТ.

Об'єктом роботи є вебсистема для автоматизованого обліку, аналізу та прогнозування витрат користувача на цифрові підписки.

Метою роботи є проєктування та реалізація вебсистеми, що дозволяє вести облік підписок вручну, автоматично виявляти регулярні списання з банківських транзакцій, прогнозувати майбутні витрати та надсилати попередні сповіщення через Telegram-бота.

Методи розроблення: тришарова архітектура REST API (роути — контролери — сервіси), алгоритм групування транзакцій за MCC-кодами та ключовими словами з подальшим статистичним аналізом інтервалів між списаннями, симетричне шифрування AES-256-CBC для захисту токена Monobank, хешування паролів bcrypt із salt-фактором 12, автентифікація на основі JSON Web Tokens із підтримкою примусового відкликання через поле tokenVersion. Інструменти розроблення: середовище виконання Node.js 18, фреймворк Express 4, СКБД MongoDB, бібліотека Mongoose 8, Telegraf 4, axios, node-cron, multer, csv-parse, xlsx. Клієнтська частина реалізована засобами HTML5, CSS3 та JavaScript без використання зовнішніх фреймворків.

Результати роботи: реалізовано REST API з 17 маршрутами, алгоритм автоматичної детекції підписок на основі банківських транзакцій, модуль прогнозування витрат на термін до 12 місяців, Telegram-бот із системою сповіщень, модуль імпорту виписок Monobank та ПриватБанку у форматах CSV, XLS та XLSX, а також односторінковий вебінтерфейс для управління підписками.

Система може застосовуватися приватними користувачами для контролю витрат на цифрові сервіси, насамперед у поєднанні з API необанку Monobank. Розроблена система може слугувати основою для ширших рішень у сфері аналізу особистих фінансів.

Робота є самостійним індивідуальним завданням і не є частиною комплексних досліджень чи спільного проєкту. Впровадження результатів у виробничому середовищі в межах даної роботи не передбачалося; система розгорнута у хмарному середовищі Railway для демонстраційних цілей.

Практична значущість роботи полягає у реалізації єдиного безкоштовного вебзастосунку, що поєднує автоматичну детекцію підписок через Monobank API з гнучким ручним обліком, — це відповідає реальній потребі українських користувачів, не забезпеченій існуючими рішеннями. Перспективами розвитку є: додавання автоматизованих тестів, підтримка пагінації, заміна обмеження частоти запитів у пам'яті на Redis, розширення переліку підтримуваних банків через стандарт відкритого банкінгу НБУ, реалізація мобільного клієнта.

ЗМІСТ

Скорочення та умовні позначення.....	5
Вступ.....	6
1 Аналіз предметної області та постановка задачі.....	9
1.1 Огляд існуючих рішень для обліку цифрових підписок.....	9
1.2 API Monobank та формати банківських виписок.....	10
1.3 Вимоги до системи.....	11
1.4 Вибір технологічного стеку та обґрунтування.....	12
2 Проєктування системи.....	14
2.1 Архітектура застосунку.....	14
2.2 Структура бази даних.....	15
2.3 Проєктування REST API.....	17
2.4 Алгоритм автоматичної детекції підписок.....	19
3 Реалізація та тестування.....	21
3.1 Реалізація серверної частини.....	21
3.2 Інтеграція з Monobank API та імпорт CSV/XLS/XLSX.....	22
3.3 Система сповіщень через Telegram-бота.....	23
3.4 Вебінтерфейс.....	23
3.5 Безпека системи.....	24
3.6 Тестування.....	25
Висновки.....	27
Перелік джерел посилання.....	29

СКОРОЧЕННЯ ТА УМОВНІ ПОЗНАЧЕННЯ

БД — база даних;

СКБД — система керування базами даних;

AES — Advanced Encryption Standard, стандарт симетричного шифрування;

API — Application Programming Interface, прикладний програмний інтерфейс;

CBC — Cipher Block Chaining, режим шифрування зі зчепленням блоків;

CSV — Comma-Separated Values, текстовий формат для табличних даних;

HTTP — HyperText Transfer Protocol, протокол передачі гіпертексту;

JSON — JavaScript Object Notation, текстовий формат обміну даними;

JWT — JSON Web Token, стандарт автентифікаційних токенів;

MCC — Merchant Category Code, код категорії торговця;

MD5 — Message Digest Algorithm 5, алгоритм хешування для генерації ідентифікаторів;

REST — Representational State Transfer, архітектурний стиль побудови розподілених застосунків;

SPA — Single-Page Application, односторінковий вебзастосунок;

XLS, XLSX — формати файлів електронних таблиць Microsoft Excel.

ВСТУП

Оцінка сучасного стану об'єкта розроблення. Впродовж останнього десятиліття ринок цифрових підписок зазнав стрімкого зростання. За даними аналітичної компанії Research and Markets, обсяг глобального ринку підписної економіки становив 623,61 млрд доларів США у 2025 році та, за прогнозами, зросте до 738,82 млрд у 2026 році [1]. Стрімінгові сервіси, хмарні середовища, SaaS-інструменти (Software as a Service — програмне забезпечення як послуга), музичні платформи та ігрові сервіси стали невіддільною частиною цифрового побуту мільйонів людей. Водночас зростання кількості підписок ускладнює їх контроль: за даними дослідження компанії C+R Research, споживачі в середньому оцінюють власні щомісячні витрати на підписки у 86 доларів, тоді як реальна сума сягає 219 доларів — тобто у 2,5 рази більше, при цьому 89% споживачів систематично недооцінюють ці витрати [2].

Особливістю українського ринку є наявність відкритого API у Monobank — одного з найбільших необанків країни, — що відкриває можливості для автоматизованого аналізу витрат на підписки безпосередньо на основі реальних банківських транзакцій.

Актуальність роботи та підстави для її виконання. Існуючі рішення для обліку підписок поділяються на два класи. Перший клас — ручні трекери (Bobby, Subby та ін.) — підтримують довільні валюти та не потребують підключення до банку, однак не мають жодної автоматичної детекції підписок з виписок. Другий клас — автоматичні трекери з банківською інтеграцією (Rocket Money, Monarch Money, PocketGuard та ін.) — виявляють підписки автоматично, але підключаються виключно до банків США та Канади через агрегатор Plaid і не підтримують інтеграцію з українськими банками. Таким чином, спеціалізований безкоштовний інструмент, який поєднував би автоматичну детекцію підписок безпосередньо з Monobank API та ручне ведення обліку в

єдиному вебзастосунку, на ринку відсутній. Це зумовлює актуальність розроблення системи, адаптованої до українських реалій.

Мета й завдання роботи. Метою курсової роботи є проєктування та реалізація вебсистеми для автоматизованого обліку, аналізу та прогнозування витрат на цифрові підписки з підтримкою Monobank API, імпорту банківських виписок у форматах CSV, XLS та XLSX, а також системи сповіщень через Telegram-бота.

Для досягнення цієї мети поставлено такі завдання:

- проаналізувати існуючі рішення для обліку підписок та визначити їх недоліки;
- сформулювати функціональні та нефункціональні вимоги до системи;
- обрати та обґрунтувати технологічний стек розроблення;
- спроектувати архітектуру системи, структуру бази даних та REST API;
- розробити алгоритм автоматичної детекції підписок із банківських транзакцій;
- реалізувати серверну частину, інтеграцію з Monobank API, модуль імпорту CSV/XLS/XLSX та Telegram-бота;
- забезпечити захист даних: шифрування чутливих полів, хешування паролів, обмеження частоти запитів;
- реалізувати клієнтську частину у вигляді односторінкового вебзастосунку;
- провести тестування основних функцій системи.

Об'єкт, методи й засоби розроблення. Об'єктом розроблення є вебсистема для обліку та аналізу підписок. Застосовано тришарову архітектуру (роути — контролери — сервіси), алгоритм групування транзакцій за МСС-кодами та ключовими словами, статистичний метод визначення інтервалу

між повторними списаннями на основі середнього арифметичного різниць дат.
Основні інструменти: Node.js, Express, MongoDB, Telegraf, Monobank API.

Можливі сфери застосування. Розроблена система може застосовуватись приватними користувачами для контролю витрат на цифрові сервіси, а також слугувати основою для ширших рішень у сфері аналізу особистих фінансів.

Взаємозв'язок з іншими роботами. Курсова робота виконана як самостійне індивідуальне завдання і не є частиною комплексних досліджень чи спільного проекту.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Огляд існуючих рішень для обліку цифрових підписок

Ринок застосунків для обліку підписок можна розділити на два основні класи: ручні трекари та автоматичні системи з банківською інтеграцією.

До першого класу належать мобільні застосунки Bobby та Subby. Bobby (iOS) — мінімалістичний трекер підписок із рейтингом 4,7 зірки у App Store; підписки вносяться вручну, підтримується одноразова покупка для розблокування додаткових можливостей [3]. Subby (iOS та Android) — безкоштовний трекер із підтримкою необмеженої кількості підписок та нагадуваннями перед списанням; безкоштовна версія показує рекламу, одноразова покупка в застосунку її прибирає [4]. Обидва застосунки підтримують довільні валюти, зокрема гривню, та не потребують підключення до банківського рахунку. Перевагою цього класу є конфіденційність даних та можливість використання в будь-якій країні. Суттєвим недоліком є відсутність автоматичного виявлення підписок: усі записи вносяться виключно вручну, що вимагає від користувача постійної дисципліни та призводить до пропуску підписок.

До другого класу належать автоматичні трекари з банківською інтеграцією: Rocket Money, Monarch Money, PocketGuard та інші. Вони підключаються до банківського рахунку через агрегатор Plaid, автоматично аналізують транзакції та виявляють регулярні списання [5]. Ці системи забезпечують повноцінну автоматичну детекцію підписок і надають детальну аналітику витрат. Критичним обмеженням є те, що Plaid підтримує виключно банки США та Канади, тому жоден із зазначених застосунків не може інтегруватись з українськими банками. Крім того, більшість цих рішень є платними або доступними лише для резидентів США.

Таким чином, на ринку відсутній безкоштовний спеціалізований вебзастосунок, який би поєднував автоматичну детекцію підписок безпосередньо з API Monobank, ручне ведення обліку, прогнозування витрат та систему сповіщень в єдиному інтерфейсі. Саме цю прогалину і закриває розроблювана система.

1.2 API Monobank та формати банківських виписок

Monobank надає відкритий REST API для доступу до інформації про рахунок та транзакції [6]. Для авторизації використовується персональний токен, який користувач генерує самостійно на вебсайті api.monobank.ua. Токен надає доступ до маршрутів GET /personal/client-info (інформація про клієнта) та GET /personal/statement/{account}/{from}/{to} (виписка за рахунком).

Запит до виписки повертає масив транзакцій за заданий часовий діапазон. Кожна транзакція містить такі поля, що є корисними для виявлення підписок:

- id — унікальний ідентифікатор транзакції;
- time — час транзакції у форматі Unix timestamp;
- amount — сума у копійках (від’ємне значення для витрат);
- mcc — код категорії торговця (Merchant Category Code);
- description — опис платежу, зазвичай містить назву сервісу;
- counterName — назва отримувача платежу, для сервісів Google та Apple є більш точною за description.

API Monobank накладає два важливі обмеження: не більше одного запиту за 60 секунд для одного рахунку, а також максимальний діапазон дат 31 день на один запит.

Окрім API, система підтримує імпорт банківських виписок у форматах CSV, XLS та XLSX. Реалізовано адаптери для двох форматів: Monobank

CSV/XLS (стовпці: «Дата і час операції», «Деталі операції», «МСС», «Сума в валюті картки (UAH)») та ПриватБанк XLSX (стовпці: «Дата», «Категорія», «Картка», «Опис операції», «Сума в валюті картки», «Валюта картки»). Формат файлу визначається автоматично на основі назв стовпців після парсингу.

Варто зазначити, що автоматична детекція підписок через API підтримується виключно для Monobank. ПриватБанк не надає відкритого токен-базованого API для фізичних осіб, аналогічного Monobank API, тому для нього реалізовано виключно імпорт через файл виписки [7].

1.3 Вимоги до системи

На підставі аналізу предметної галузі та вивчення аналогів сформульовано вимоги до системи.

Функціональні вимоги:

- реєстрація та автентифікація користувача за електронною поштою та паролем;
- ручне додавання, редагування, видалення та призупинення підписок;
- інтеграція з Monobank API: збереження зашифрованого токена, синхронізація транзакцій за 90 днів, автоматична детекція підписок;
- імпорт банківських виписок у форматах CSV, XLS та XLSX (Monobank та ПриватБанк) з автоматичною детекцією підписок;
- прогнозування витрат на наступні 3, 6 або 12 місяців у розрізі місяців та валют;
- надсилання Telegram-сповіщень про майбутні списання за налаштовану кількість днів наперед;
- Telegram-бот із командами /subscriptions та /upcoming;

— перегляд підписок із фільтрацією за статусом (активна/призупинена) та категорією.

Нефункціональні вимоги:

— зберігання токена Monobank у зашифрованому вигляді (AES-256-CBC);

— хешування паролів bcrypt із salt-фактором 12;

— JWT-автентифікація з підтримкою відкликання токенів через tokenVersion;

— обмеження частоти запитів на маршрути автентифікації (10 спроб за 15 хвилин з однієї IP-адреси) та синхронізації (3 запити за 10 хвилин);

— валідація вхідних даних на рівні проміжних обробників (middleware) перед передачею у контролер;

— централізована обробка помилок із розпізнаванням помилок валідації Mongoose, конфліктів унікального ключа, помилок JWT та відповідей 429 від зовнішніх API.

1.4 Вибір технологічного стеку та обґрунтування

При виборі технологічного стеку враховувались такі критерії: єдина мова програмування для сервера та клієнта, зрілість екосистеми, наявність готових бібліотек для роботи з Monobank API та Telegram Bot API, а також простота розгортання.

Node.js забезпечує єдину мову (JavaScript) для серверної та клієнтської частин системи. Асинхронна модель виконання на основі циклу подій є природною для застосунків, що виконують численні мережеві запити — до Monobank API, MongoDB та Telegram API [8].

Express — мінімалістичний фреймворк для Node.js, стандарт де-факто для побудови REST API [9]. Він не нав'язує структури застосунку, що дозволяє організувати код у тришарову архітектуру. Система проміжних обробників Express дозволяє легко підключати Helmet, Morgan та власні обробники запитів.

MongoDB — документоорієнтована СКБД, що зберігає дані у форматі BSON. Для підписок вона є природним вибором, оскільки модель даних підписки має опціональні поля (наприклад, `trialEndDate`, `reminderDays`, `customCycleDays`), які не є обов'язковими для всіх записів. Mongoose надає схеми, валідацію та зручний інтерфейс поверх драйвера MongoDB [10].

Telegraf — типізований фреймворк для Telegram Bot API на Node.js [11]. Він надає зручний інтерфейс для обробки команд та підтримку режиму тривалого опитування (long polling) — техніки взаємодії з сервером Telegram, за якої бот утримує HTTP-з'єднання відкритим до надходження нового повідомлення або до закінчення часу очікування, після чого відкриває нове. Цей підхід не потребує публічної HTTPS-адреси і є зручним для розроблення.

Клієнтська частина реалізована на основі HTML5, CSS3 та JavaScript без використання зовнішніх фреймворків. Такий підхід зменшує кількість залежностей та спрощує розгортання: клієнтська частина не потребує окремого кроку збірки і роздається безпосередньо Express-сервером через `express.static()`.

2 ПРОЄКТУВАННЯ СИСТЕМИ

2.1 Архітектура застосунку

Система побудована за тришаровою архітектурою. Перший шар — роути — відповідає за визначення HTTP-маршрутів та прив'язку до проміжних обробників. Другий шар — контролери та сервіси — реалізує бізнес-логіку: контролер отримує оброблений запит від роуту і делегує логіку відповідному сервісу. Третій шар — моделі Mongoose — забезпечує роботу з базою даних. Така організація знижує зв'язність компонентів: контролери не знають деталей роботи з БД, сервіси не знають деталей HTTP-протоколу.

Серверна частина реалізована як Node.js-процес, що одночасно виконує: обробку HTTP-запитів через Express, роздачу статичних файлів клієнтської частини, підключення до MongoDB через Mongoose, запуск Telegram-бота в режимі тривалого опитування через Telegraf, та запуск планувальника щоденних сповіщень через node-cron.

Файли застосунку організовано за функціональною ознакою:

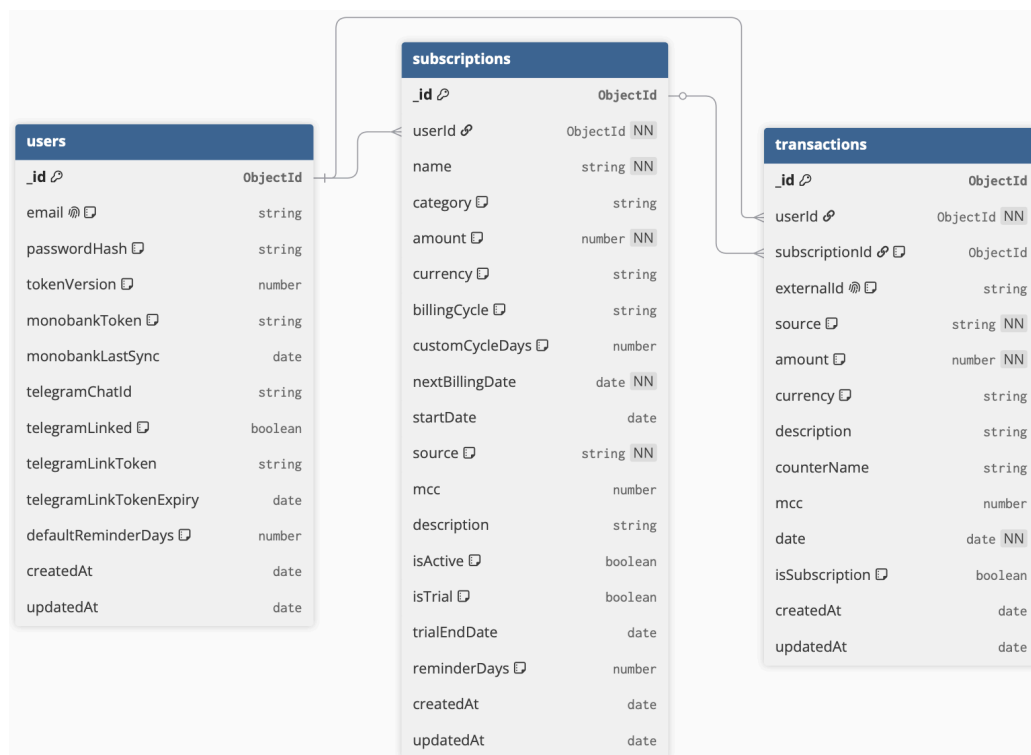
- `src/routes/` — визначення HTTP-маршрутів (`auth.js`, `subscriptions.js`, `monobank.js`, `import.js`, `forecast.js`);
- `src/controllers/` — обробники запитів (`authController.js`, `subscriptionController.js`, `monobankController.js`, `importController.js`, `forecastController.js`);
- `src/services/` — бізнес-логіка (`monobankService.js`, `subscriptionDetector.js`, `forecastService.js`, `notificationService.js`, `cryptoService.js`);
- `src/models/` — схеми Mongoose (`User.js`, `Subscription.js`, `Transaction.js`);
- `src/middleware/` — `auth.js`, `rateLimit.js`, `validate.js`, `errorHandler.js`;

- `src/bot/` — Telegram-бот та його команди (`commands/start.js`, `subscriptions.js`, `upcoming.js`);
- `src/utils/` — допоміжні функції (`csvParser.js`, `dateUtils.js`);
- `public/` — статичний вебклієнт (`index.html`, `app.js`, `style.css`).

Точка входу `index.js` перевіряє наявність обов'язкових змінних середовища (`JWT_SECRET`, `CRYPTO_SECRET`, `MONGODB_URI`) до ініціалізації будь-яких підключень. За їх відсутності процес завершується з описовим повідомленням про помилку. Це запобігає запуску системи з неповною конфігурацією.

2.2 Структура бази даних

База даних містить три колекції (рисуюнок 2.1): `User`, `Subscription` та `Transaction`.



Рисуюнок 2.1 — Структура бази даних системи обліку підписок

Колекція User зберігає облікові дані та налаштування користувача. Поле passwordHash містить bcrypt-хеш пароля (salt-фактор 12). Поле tokenVersion використовується для примусового відкликання всіх JWT-токенів користувача: при кожному збільшенні цього поля всі раніше видані токени стають недійсними. Поля monobankToken (зашифрований AES-256-CBC токен Monobank) та telegramChatId/telegramLinked забезпечують інтеграцію із зовнішніми сервісами. Поле defaultReminderDays визначає стандартний час нагадування (за замовчуванням 3 дні).

Колекція Subscription зберігає інформацію про підписку. Поле billingCycle приймає значення weekly, monthly, yearly або custom (для нестандартних циклів тривалість у днях зберігається в customCycleDays). Поле source фіксує спосіб додавання: manual — вручну, monobank — через API, csv — через імпорт файлу (CSV, XLS, XLSX). Поле nextBillingDate є основою для прогнозування і розрахунку дати сповіщення. Поле reminderDays, якщо задане, перевизначає defaultReminderDays користувача для конкретної підписки. Поле category може приймати значення entertainment, work, cloud, music, gaming або other.

Колекція Transaction зберігає транзакції, отримані від API, або завантажені через CSV/XLS/XLSX. Поле externalId містить унікальний ідентифікатор транзакції з джерела. Для CSV-файлів він генерується як хеш MD5 від конкатенації дати, суми та опису транзакції — це гарантує унікальність і запобігає дублюванню при повторному завантаженні одного файлу. MD5 використовується виключно для генерації ідентифікатора дедуплікації, а не з криптологічною метою. Значення суми зберігається в копійках відповідно до формату Monobank API. Для поля externalId у Mongoose задано sparse-індекс: він гарантує унікальність непустих значень, але допускає зберігання null. Це дозволяє використовувати одну колекцію для транзакцій з різних джерел без конфліктів.

2.3 Проектування REST API

API побудовано за ресурсним принципом REST. Усі маршрути, крім POST /api/auth/register, POST /api/auth/login та GET /api/health, вимагають заголовка Authorization: Bearer <token>. Перелік реалізованих маршрутів представлено в таблиці 2.1.

Таблиця 2.1 — Перелік маршрутів REST API

Метод	URL	Призначення	Захист
POST	/api/auth/register	Реєстрація	Публічний
POST	/api/auth/login	Вхід в систему	Публічний
GET	/api/auth/me	Дані поточного користувача	JWT
POST	/api/auth/link-token	Генерація токена для Telegram	JWT
POST	/api/auth/revoke-tokens	Відкликання всіх JWT	JWT
GET	/api/subscriptions	Список підписок	JWT
POST	/api/subscriptions	Створення підписки	JWT
PUT	/api/subscriptions/:id	Оновлення підписки	JWT
DELETE	/api/subscriptions/:id	Видалення підписки	JWT
PATCH	/api/subscriptions/:id/toggle	Зміна статусу активності	JWT
POST	/api/monobank/token	Збереження токена Monobank	JWT

Продовження таблиці 2.1

Метод	URL	Призначення	Захист
DELETE	/api/monobank/token	Видалення токена Monobank	JWT
POST	/api/monobank/sync	Синхронізація транзакцій	JWT та Rate limit
GET	/api/monobank/status	Статус підключення Monobank	JWT
POST	/api/import	Імпорт CSV/XLS/XLSX виписки	JWT
GET	/api/forecast	Прогноз витрат	JWT
GET	/api/health	Перевірка стану сервера	Публічний

Типовий ланцюжок проміжних обробників для захищеного маршруту: auth (перевірка JWT, завантаження User з БД з виключенням passwordHash) — validate (перевірка тіла запиту) — controller — errorHandler. Централізований обробник помилок розпізнає: помилки валідації Mongoose (HTTP 400), конфлікти унікального ключа MongoDB (код 11000, HTTP 409), помилки JWT (HTTP 401) та відповіді 429 від зовнішніх API.

2.4 Алгоритм автоматичної детекції підписок

Алгоритм детекції підписок із транзакцій складається з трьох послідовних етапів (рисунок 2.2).

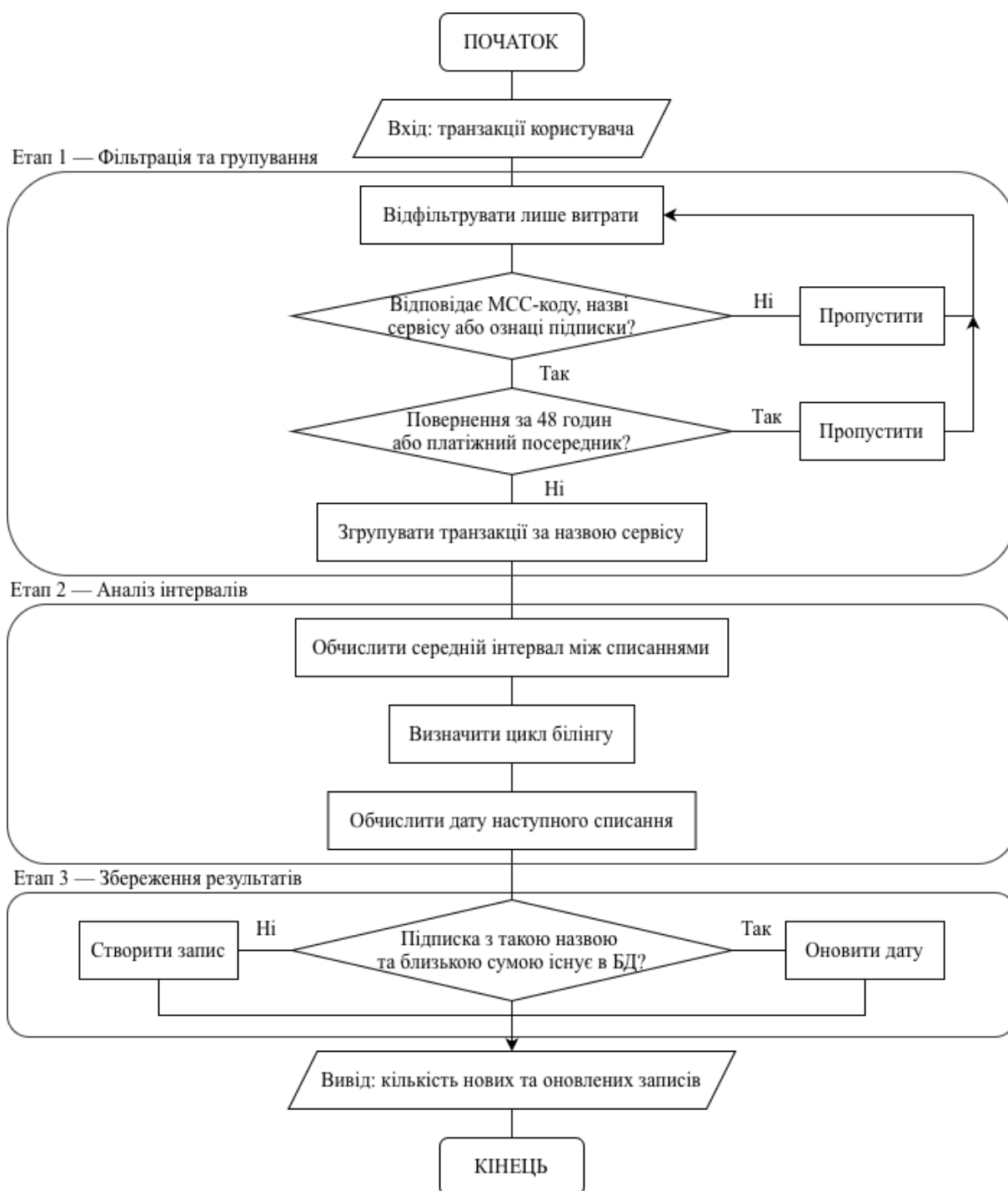


Рисунок 2.2 — Схема алгоритму автоматичної детекції підписок

Етап 1. Фільтрація та групування. З набору транзакцій відфільтровуються лише витрати (від’ємна сума). Для кожної транзакції виконується класифікація за трьома критеріями: MCC-код транзакції належить до списку підписних кодів (5815, 5817, 7372, 4816, 4899, 7995, 5968); поле `counterName` або `description` містить назву відомого сервісу (Netflix, Spotify, YouTube, Apple, Google та ін.); поле містить загальну ознаку підписки (`subscription`, `premium`, `monthly`, підписка, преміум тощо). Транзакції, що пройшли фільтр, групуються за ключем: для відомих сервісів — за назвою, для невідомих — за комбінацією MCC-коду, заокругленої суми та назви. Платіжні посередники та магазини з разовими покупками (Steam, Epic Games Store тощо) виключаються через список `NON_SUBSCRIPTION_NAMES` незалежно від MCC-коду. Також виключаються разові пробні або верифікаційні списання, які були повернуті протягом 48 годин (наприклад, тестові списання Google для перевірки картки).

Етап 2. Аналіз інтервалів. Для кожної групи з двома і більше транзакціями обчислюється середнє арифметичне міжчасових інтервалів між послідовними транзакціями. На основі цього значення визначається цикл білінгу: менше 9 днів — `weekly`, від 9 до 100 днів — `monthly`, від 100 до 400 днів — `yearly`, понад 400 днів — `custom` зі збереженням точної тривалості. Для груп з однією транзакцією від відомого сервісу за замовчуванням приймається місячний цикл.

Етап 3. Збереження результатів. Для кожної групи система перевіряє, чи існує вже підписка з такою ж назвою та близькою сумою (у межах плюс-мінус 10%). Якщо підписки немає — вона створюється з джерелом `monobank` або `csv` залежно від походження транзакцій. Якщо існує — оновлюється лише `nextBillingDate`. Дата наступного списання обчислюється функцією `calcNextBillingDate` на основі останньої відомої дати та визначеного циклу. Для місячного та річного циклів дата додатково прив’язується до дня місяця з найстарішої транзакції групи, що запобігає поступовому зміщенню очікуваної дати списання при незначних затримках платежу.

3 РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ

3.1 Реалізація серверної частини

Серверна частина реалізована на Node.js 18 з фреймворком Express 4. Ініціалізація застосунку відбувається в `index.js`: перевіряється наявність обов'язкових змінних середовища (`JWT_SECRET`, `CRYPTO_SECRET`, `MONGODB_URI`), після чого послідовно виконується підключення до MongoDB, запуск HTTP-сервера, запуск Telegram-бота та планувальника сповіщень. Двічі при запуску викликається `advancePastDates()`: всередині `startScheduler()` та явно в `index.js` після її виклику — це гарантує оновлення застарілих дат `nextBillingDate`, не чекаючи першого спрацювання стоп-задачі.

Проміжний обробник `helmet()` встановлює захисні заголовки HTTP (`X-Frame-Options`, `Content-Security-Policy` та ін.) [12]. `cors()` налаштовується через змінну середовища `CLIENT_URL`. `morgan()` веде лог HTTP-запитів у режимі `dev`. Статичні файли клієнтської частини роздаються через `express.static()` з директорії `public/`.

Автентифікація реалізована через JWT-токени з терміном дії 30 днів [13]. Токен містить поля `id` та `tokenVersion` користувача. При кожному автентифікованому запиті проміжний обробник `auth.js` завантажує з БД актуальний об'єкт `User` (виключаючи `passwordHash`) і порівнює `tokenVersion` у токені з поточним значенням у БД. Невідповідність означає, що токен відкликано через `POST /api/auth/revoke-tokens` — запит відхиляється зі статусом HTTP 401.

Обмеження частоти запитів (`rate limiting`) реалізоване в пам'яті процесу: для кожної IP-адреси зберігається масив часових міток запитів. Прострочені записи видаляються кожні 10 хвилин для запобігання витокам пам'яті. На

маршрути реєстрації та входу встановлено ліміт 10 спроб за 15 хвилин; на маршрут синхронізації Monobank — 3 запити за 10 хвилин.

3.2 Інтеграція з Monobank API та імпорт CSV/XLS/XLSX

При збереженні токена Monobank виконується його верифікація через GET /personal/client-info. У разі успіху токен шифрується алгоритмом AES-256-CBC з випадковим 16-байтним вектором ініціалізації (IV) перед кожним шифруванням. Зашифрований рядок зберігається у форматі hex(IV):hex(ciphertext). При синхронізації токен розшифровується безпосередньо перед запитом до API.

Синхронізація транзакцій охоплює 90 днів. Оскільки Monobank дозволяє максимум 31 день на запит та 1 запит за 60 секунд, функція fetchTransactions() розбиває діапазон на частини по 31 день і надсилає запити послідовно з паузою 61 секунда між ними. Транзакції зберігаються операцією вставки з оновленням (upsert) за полем externalId, що гарантує ідемпотентність: повторна синхронізація не дублює дані.

Модуль імпорту (csvParser.js) підтримує три формати файлів. Для CSV використовується бібліотека csv-parse [14]. Для XLS та XLSX — бібліотека SheetJS (xlsx) [15]. Особливістю файлів Monobank XLS є наявність блоку метаданих клієнта (ім'я, ПІН, номер рахунку) перед рядком заголовків таблиці: алгоритм автоматично знаходить рядок заголовків за наявністю не менше п'яти непустих стовпців. Виявлення адаптера (Monobank або ПриватБанк) відбувається автоматично за назвами стовпців після парсингу. Підтримуються як україномовні, так і англomовні версії виписок Monobank. Кодування символів нормалізується до NFC для коректного порівняння рядків, оскільки XLS-файли Monobank зберігають кирилицю у форматі NFD.

3.3 Система сповіщень через Telegram-бота

Telegram-бот реалізований за допомогою фреймворку Telegraf 4 в режимі тривалого опитування (long polling). Бот підтримує три команди: `/start` (прив'язка облікового запису), `/subscriptions` (список активних підписок) та `/upcoming` (майбутні списання протягом наступних 7 днів).

Прив'язка облікового запису реалізована через одноразовий токен: користувач генерує 32-символьний hex-токен у вебзастосунку, він зберігається в БД з терміном дії 15 хвилин. При відправці команди `/start <token>` бот знаходить відповідний обліковий запис, очищує `chatId` у будь-якого іншого облікового запису, де він вже використовувався (запобігання «забруднення» БД), та прив'язує поточний `chatId`.

Щоденні сповіщення реалізовані через `node-cron` [16]: планувальник запускається щодня о 09:00 (час налаштовується через змінні середовища `NOTIFICATION_HOUR` та `NOTIFICATION_MINUTE`). Для кожної активної підписки обчислюється дата сповіщення як `nextBillingDate` мінус `reminderDays` (або `defaultReminderDays` користувача). Якщо поточна дата збігається з датою сповіщення, бот надсилає повідомлення у форматі `MarkdownV2` із назвою підписки, датою та сумою. Щоденно також виконується `advancePastDates()` — функція, що автоматично просуває `nextBillingDate` вперед для підписок із застарілими датами. Усі рядки, що вставляються у повідомлення, проходять через функцію екранування спеціальних символів відповідно до специфікації Telegram Bot API.

3.4 Вебінтерфейс

Клієнтська частина реалізована як SPA на основі HTML5, CSS3 та JavaScript без зовнішніх фреймворків. Навігація між сторінками реалізована

через функцію `go()`, що показує або приховує відповідні блоки без перезавантаження сторінки. JWT-токен зберігається в `localStorage` та передається з кожним запитом у заголовок `Authorization: Bearer`.

Інтерфейс містить п'ять розділів: «Огляд» (Dashboard) — зведена статистика, найближче списання, підписки в межах 7 днів; «Підписки» (Subscriptions) — повний список з можливістю додавання, редагування, видалення та призупинення; «Прогноз» (Forecast) — прогноз витрат на 3/6/12 місяців із поділом за валютами та повним списком майбутніх списань; «Імпорт» (Import) — завантаження файлів Monobank/ПриватБанку та підключення Monobank API; «Налаштування» (Settings) — прив'язка Telegram та налаштування нагадувань.

Функція `signOut()` при виході очищує JWT-токен з `localStorage`, скидає стан DOM (повідомлення про результат імпорту, статус підключення Monobank тощо) та перемикає інтерфейс на екран входу. Це гарантує, що при повторному вході в інший обліковий запис залишки стану попереднього сеансу не відображаються.

3.5 Безпека системи

Захист чутливих даних реалізовано на кількох рівнях. Паролі зберігаються виключно у вигляді bcrypt-хешу із salt-фактором 12, що відповідає рекомендаціям щодо захисту від атак перебору [17]. Токен Monobank шифрується AES-256-CBC з унікальним вектором ініціалізації для кожного шифрування, що унеможливорює атаки на основі аналізу шаблонів шифротексту.

JWT-токени містять поле `tokenVersion`, що дозволяє миттєво відкликати всі сесії користувача через `POST /api/auth/revoke-tokens`. Обмеження частоти запитів на рівні проміжних обробників захищає від атак перебором (brute-force) на маршрути автентифікації.

Усі запити до захищених маршрутів перевіряють `userId` виключно з поля `req.user` (завантаженого з БД після верифікації JWT), а не з тіла запиту. Це запобігає атакам типу небезпечного прямого звернення до об'єктів (Insecure Direct Object Reference, IDOR). При операціях з підписками умова завжди включає `userId: req.user._id` — користувач може читати та модифікувати лише свої дані.

3.6 Тестування

Тестування основних функцій системи проводилось вручну з використанням `curl` та веббраузера. Перевірялися такі сценарії:

- реєстрація нового користувача та вхід в систему; перевірка відхилення невалідних електронних адрес та коротких паролів;
- спроба доступу до захищеного маршруту без токена та з відкликаним токеном;
- створення підписок вручну із різними циклами (`weekly`, `monthly`, `yearly`, `custom`); перевірка коректності `calcNextBillingDate`;
- імпорт реальних виписок Monobank (CSV україномовний, CSV англomовний, XLS україномовний, XLS англomовний) та ПриватБанк XLSX; перевірка кількості розпізнаних транзакцій та детектованих підписок;
- перевірка прогнозу витрат: коректність дат та сум для підписок із різними циклами та валютами;
- прив'язка до Telegram-бота; перевірка команд `/subscriptions` та `/upcoming`; перевірка коректності екранування спеціальних символів у назвах підписок;
- перевірка `rate limiting`: надсилення більше 10 запитів на вхід протягом 15 хвилин — система повертає HTTP 429;

— повторний імпорт одного файлу — перевірка відсутності дублювання транзакцій.

ВИСНОВКИ

У результаті виконання курсової роботи було розроблено вебсистему для автоматизованого обліку, аналізу та прогнозування витрат на цифрові підписки, адаптовану до українського ринку та інтегровану з Monobank API.

У процесі виконання роботи було проаналізовано існуючі рішення для обліку підписок та визначено їх ключові обмеження: ручні трекери (Bobby, Subby) не забезпечують автоматичної детекції підписок, а автоматичні системи з банківською інтеграцією (Rocket Money, Monarch Money) орієнтовані виключно на ринок США та Канади і не підтримують інтеграцію з українськими банками.

Спроектовано та реалізовано тришарову серверну архітектуру на Node.js та Express із підключенням до MongoDB. Розроблено алгоритм автоматичної детекції підписок із транзакцій на основі MCC-кодів, ключових слів та статистичного аналізу інтервалів між списаннями. Алгоритм коректно виключає платіжні посередники та магазини з разовими покупками.

Реалізовано модуль імпорту банківських виписок, що підтримує формати CSV, XLS та XLSX від Monobank та ПриватБанку, включно з автоматичним визначенням формату, нормалізацією кодування (NFC) та дедуплікацією транзакцій.

Розроблено Telegram-бота з командами /subscriptions та /upcoming і системою щоденних проактивних сповіщень через node-cron. Забезпечено багаторівневий захист даних: AES-256-CBC шифрування токена Monobank, bcrypt-хешування паролів, JWT-автентифікація з примусовим відкликанням через tokenVersion, обмеження частоти запитів на критичних маршрутах.

Практична цінність системи полягає в тому, що вона вирішує реальну задачу контролю витрат на підписки в українських реаліях, поєднуючи

автоматизовану детекцію через Monobank API з гнучким ручним обліком у єдиному безкоштовному вебінтерфейсі.

Перспективами розвитку системи є додавання автоматизованих тестів (Jest); підтримка пагінації результатів для великих наборів даних; заміна обмеження частоти запитів у пам'яті на Redis для підтримки багатопроцесного розгортання; розширення списку підтримуваних банків через стандарт відкритого банкінгу Національного банку України; реалізація мобільного клієнта.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Research and Markets. Subscription Economy Market Report, 2026. URL: https://www.researchandmarkets.com/reports/6225942/subscription-economy-market-report?srsltid=AfmBOop5u1xhXn305_dnd1QdfPnfpDtnEyJXkl0Q7L2qBhJtTlGsMktP (дата звернення: 12.02.2026).
2. C+R Research. Subscription Service Statistics and Costs, 2022. URL: <https://www.crresearch.com/blog/subscription-service-statistics-and-costs/> (дата звернення: 14.02.2026).
3. Yummygum. Bobby — Track subscriptions: App Store. URL: <https://apps.apple.com/us/app/bobby-track-subscriptions/id1059152023> (дата звернення: 17.02.2026).
4. Subby — The Subscription Manager. Google Play. URL: <https://play.google.com/store/apps/details?id=com.slapp.subby&hl=uk> (дата звернення: 17.02.2026).
5. Plaid. Countries and regions supported by Plaid. URL: <https://plaid.com/global/> (дата звернення: 17.02.2026).
6. Monobank. Open API: документація для фізичних осіб та ФОП. URL: <https://api.monobank.ua/docs/index.html> (дата звернення: 05.03.2026).
7. ПриватБанк. API ПриватБанку: публічні сервіси. URL: <https://api.privatbank.ua> (дата звернення: 06.03.2026).
8. OpenJS Foundation. Node.js Documentation. URL: <https://nodejs.org/en/docs> (дата звернення: 10.03.2026).
9. Express.js. Express 4.x API Reference. URL: <https://expressjs.com/en/4x/api.html> (дата звернення: 10.03.2026).

10. MongoDB. Mongoose ODM Documentation, v8. URL: <https://mongoosejs.com/docs> (дата звернення: 12.03.2026).
11. Telegraf. Modern Telegram Bot Framework for Node.js. URL: <https://telegraf.js.org> (дата звернення: 24.03.2026).
12. Helmet.js. Help secure Express apps with various HTTP headers. URL: <https://helmetjs.github.io> (дата звернення: 02.04.2026).
13. Auth0. JSON Web Tokens (JWT) Introduction. URL: <https://jwt.io/introduction> (дата звернення: 02.04.2026).
14. csv-parse. CSV parsing library for Node.js. URL: <https://csv.js.org/parse> (дата звернення: 15.04.2026).
15. SheetJS. Community Edition: Spreadsheet Data Toolkit. URL: <https://docs.sheetjs.com> (дата звернення: 15.04.2026).
16. node-cron. Task Scheduler for Node.js. URL: <https://github.com/node-cron/node-cron> (дата звернення: 29.04.2026).
17. OWASP. Password Storage Cheat Sheet. URL: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (дата звернення: 22.05.2026).